

Payment Workflow Schematic

A production-oriented view of how Ridendine accepts customer payment as merchant of record, routes Stripe events into order operations, and records finance state through ledger, payout, refund, and reconciliation services.

Merchant Model

Ridendine is merchant of record

Primary Processor

Stripe PaymentIntent + webhooks

System of Record

Supabase + ledger_entries

Safety Pattern

Idempotency, audit, reconciliation

This document reflects current repo wiring from checkout, webhook, ledger, and payout source files. It does not claim production readiness for money movement without the remaining RBAC, replay, and reconciliation tests.

Customer Payment

- Validates customer, cart, menu prices, promo, risk, and kitchen readiness.
- Creates the order before initializing Stripe payment.
- Returns Stripe client secret and authoritative total breakdown.

Stripe Event Intake

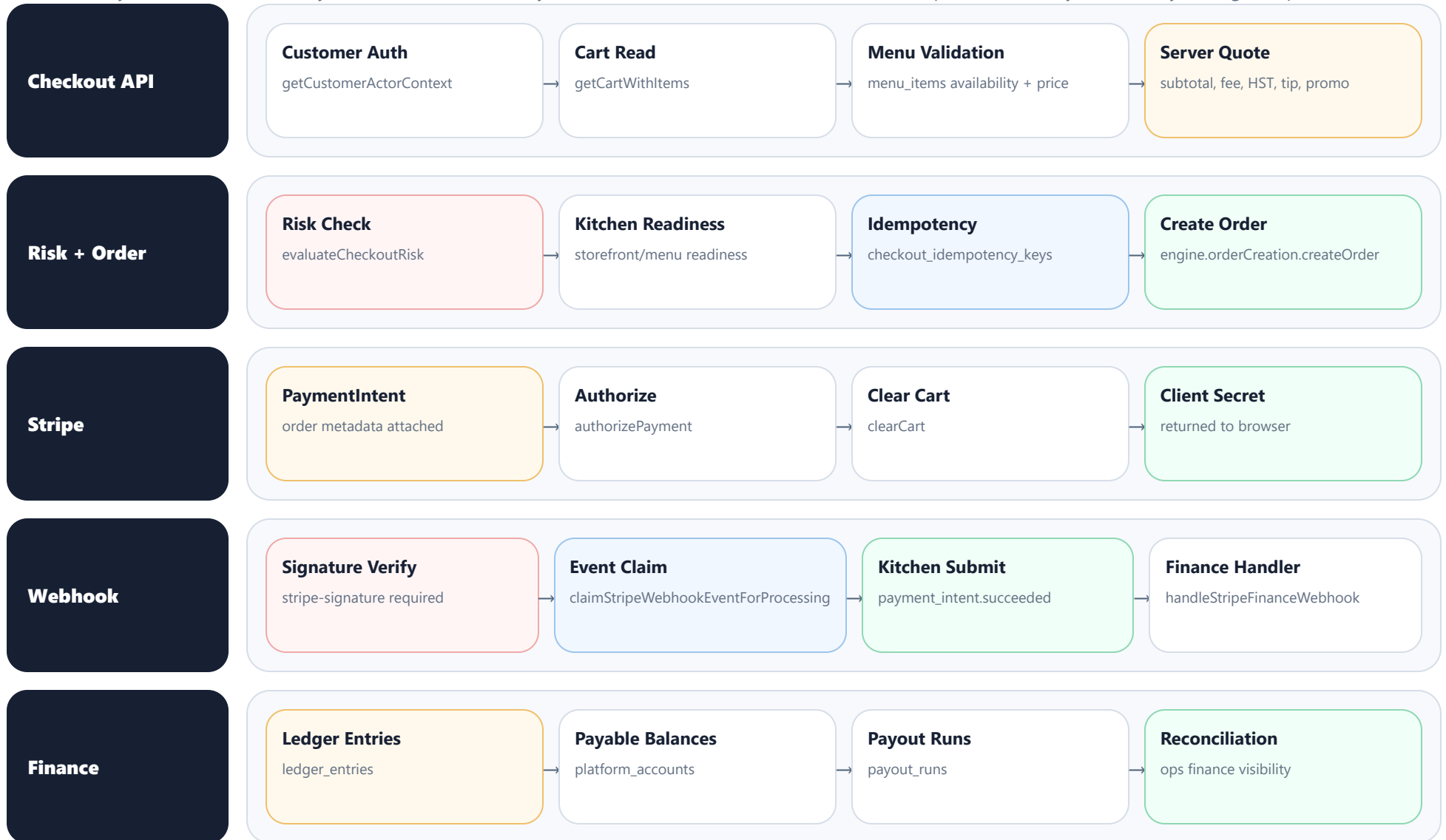
- Verifies webhook signature.
- Claims event idempotently before processing.
- Routes payment success, failure, refund, transfer, and payout events.

Finance Ledger

- Records capture, tax, platform fee, chef payable, driver payable, tips, refunds, and payout debits.
- Uses idempotency keys to avoid duplicate money entries.

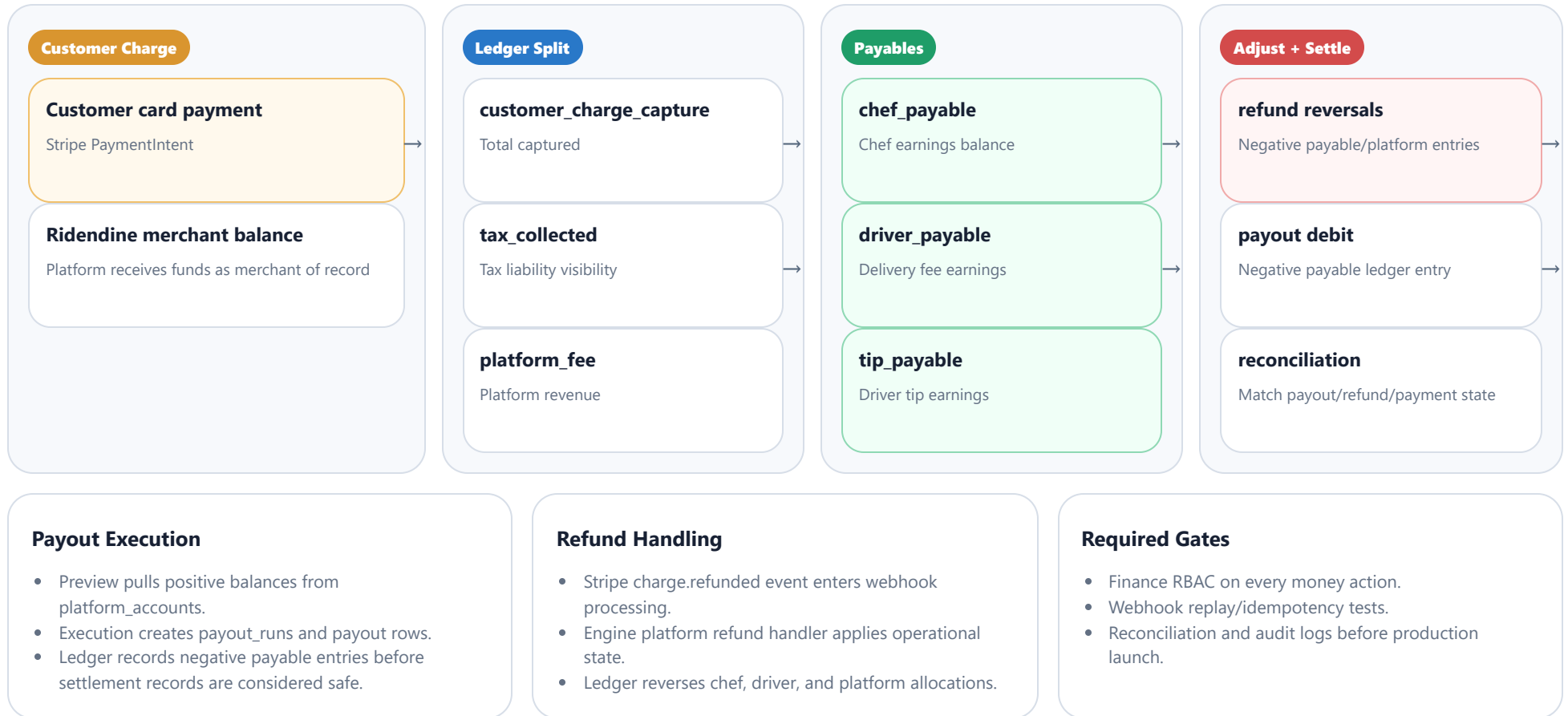
End-To-End Operational Flow

Checkout is synchronous until the PaymentIntent is created. Payment confirmation and downstream finance updates arrive asynchronously through Stripe webhooks.



Money Movement Model

Ridendine receives the customer payment, then ledger entries describe how money is allocated, reversed, and settled.



Repo Evidence And Controls

AREA	CURRENT SOURCE	WHAT IT PROVES
Checkout + PaymentIntent	apps/web/src/app/api/checkout/route.ts	Customer auth, cart/menu validation, server quote, risk, kitchen readiness, checkout idempotency, order creation, Stripe PaymentIntent creation, payment authorization.
Stripe Webhook	apps/web/src/app/api/webhooks/stripe/route.ts	Signature verification, webhook rate limit, idempotent event claim, payment success/failure/refund routing, audit logging, finance webhook handling.
Ledger	packages/engine/src/services/ledger.service.ts	Idempotent insertion into ledger_entries for capture, tax, payables, platform fee, refund reversals, payout debits, and instant payout fees.
Payouts	packages/engine/src/services/payout.service.ts	Chef/driver payout preview, payout run creation, payout risk checks, bank payout lifecycle, instant payout request/execution, payout ledger debits.
Engine Exports	packages/engine/src/index.ts	Stripe, webhook idempotency, finance webhook, ledger, payout, and reconciliation services are exported for app APIs.

Production note: the schematic describes intended/current wiring, not a launch approval. Before real money goes live, protect all finance/admin actions with role checks, add negative tests, prove Stripe webhook replay safety, and verify reconciliation can explain every dollar.